SECTION A

✅ **Q1(A). Explain the role of interrupts in OS design. Also explain types of interrupts.**

📌 **Answer:**

Interrupts are signals sent to the CPU to indicate that an event needs immediate attention. They play a crucial role in OS design by enabling efficient CPU utilization and multitasking.

🎯 **Role of Interrupts:**

- Allow CPU to respond to I/O events quickly
- Avoids **busy waiting** (CPU doesn't keep checking devices)
- Enables **multitasking and time-sharing**
- Improves system efficiency

🔹 **Types of Interrupts:**

1. Hardware Interrupts:

- Generated by hardware devices
  👉 Example: Keyboard input, disk completion

---

2. Software Interrupts:

- Generated by programs (system calls)
  👉 Example: Request for OS service

---

3. Maskable Interrupts:

- Can be disabled by CPU
- Lower priority

---

4. Non-Maskable Interrupts:

- Cannot be ignored

- Used for critical events (e.g., hardware failure)

✅ **Conclusion:**

Interrupts make OS responsive and efficient by handling events asynchronously.

✅ **Q1(B). Contiguous and Non-Contiguous Memory Allocation**

📌 **Answer:**

🔹 **Contiguous Memory Allocation:**

Each process is allocated a **single continuous block** of memory.

Advantages:

- Simple to implement
- Fast memory access

Disadvantages:

- External fragmentation
- Difficult to allocate large processes

🔹 **Non-Contiguous Memory Allocation:**

Process is divided into parts and stored in different memory locations.

Examples: Paging, Segmentation

Advantages:

- Better memory utilization
- No external fragmentation

Disadvantages:

- Complex implementation
- Requires address translation

🔥 **Differences:**

Feature	Contiguous	Non-Contiguous
Memory	Continuous	Scattered
Efficiency	Low	High
Complexity	Low	High

✅ **Q2(A). Compare file access methods.**

📌 **Answer:**

🔹 **Sequential Access:**

- Data accessed one by one in order
- Used in tapes

🔹 **Direct Access:**

- Random access to any block
- Used in disks

🔹 **Indexed Access:**

- Uses index to locate data quickly

🎯 **Comparison:**

Method	Speed	Flexibility
Sequential	Slow	Low
Direct	Fast	High
Indexed	Very Fast	High

✅ **Q2(B). Why rotational latency is usually not considered?**

📌 **Answer:**

Rotational latency is the time taken for disk to rotate to correct position.

🎯 **Reasons:**

- Smaller compared to seek time
- Often constant/average value
- Seek time dominates performance
- Simplifies scheduling calculations

OR

Second-Chance (Clock) Algorithm

📌 **Answer:**

It is an improvement over FIFO using a **reference bit**.

🎯 **Working:**

- If reference bit = 1 → give second chance
- If reference bit = 0 → replace page

✅ **Advantage:**

Reduces unnecessary page replacement compared to FIFO

✅ **Q3(A). Types of threads and functionalities**

📌 **Answer:**

🔹 **User-Level Threads:**

- Managed by user library

- Fast but less powerful

🔹 **Kernel-Level Threads:**

- Managed by OS
- Slower but more efficient

🎯 **Functions:**

- Enable parallel execution
- Improve CPU utilization
- Support multitasking

✅ **Q3(B). Role of Dirty Bit**

📌 **Answer:**

Dirty bit indicates whether a page has been modified.

🎯 **Working:**

- Dirty = 1 → modified → write to disk
- Dirty = 0 → no need to write

✅ **Benefit:**

- Reduces unnecessary disk operations
- Improves performance

OR

Device Drivers & Device-Independent I/O

📌 **Answer:**

- **Device Driver:** Software that controls hardware devices
- **Device-independent I/O:** OS-level functions like buffering, caching

SECTION B —

✅ **Q1(A). Explain what is memory management in an operating system. Why is it important? Define logical and physical address.**

📌 **Answer:**

🔹 **What is Memory Management?**

Memory management is the function of an operating system that **controls and manages main memory (RAM)** by allocating and deallocating memory to processes.

🔹 **Why is Memory Management Important?**

1. **Efficient Utilization**
   → Ensures memory is used properly without wastage
2. **Process Isolation**
   → Prevents one process from interfering with another
3. **Multitasking Support**
   → Allows multiple programs to run simultaneously
4. **Memory Protection**
   → Protects data from unauthorized access

🔹 **Logical Address vs Physical Address:**

Feature	Logical Address	Physical Address
Generated by	CPU	Memory unit
Visibility	User/program	Hardware
Conversion	Converted by MMU	Final location

✅ **Conclusion:**

Memory management ensures **efficient, safe, and organized use of memory**.

✅ **Q1(B). Describe the generations of operating systems.**

📌 **Answer:**

🔹 **1st Generation (1940–1950)**

- No operating system
- Programs executed manually
- Used vacuum tubes

🔹 **2nd Generation (1950–1960)**

- Batch processing systems
- Jobs executed sequentially

🔹 **3rd Generation (1960–1980)**

- Multiprogramming introduced
- Time-sharing systems
- Better CPU utilization

🔹 **4th Generation (1980–Present)**

- Personal computers
- GUI-based systems (Windows, Linux)

🔹 **5th Generation (Future)**

- AI-based systems
- Cloud computing
- Distributed systems

✅ **Conclusion:**

OS evolved from manual systems to **intelligent and automated systems**.

✅ **Q2(A). Understand the role of semaphores in synchronization. How do semaphores prevent race conditions in the producer-consumer problem?**

📌 **Answer:**

🔹 **What is a Semaphore?**

A semaphore is a **synchronization tool** used to control access to shared resources.

🔹 **Types:**

- Binary semaphore (0 or 1)
- Counting semaphore

🔹 **Role in Synchronization:**

- Ensures **mutual exclusion**
- Controls process execution order
- Prevents data inconsistency

🔹 **In Producer-Consumer Problem:**

Three semaphores are used:

- mutex → ensures one process at a time
- empty → tracks empty buffer slots
- full → tracks filled slots

🔹 **How Race Condition is Prevented:**

- Producer waits if buffer is full
- Consumer waits if buffer is empty

- Only one process accesses buffer at a time

✅ **Conclusion:**

Semaphores ensure **safe and synchronized access to shared resources**.

✅ **Q2(B). Explain the concept of a hashed page table and its working mechanism.**

📌 **Answer:**

🔹 **Definition:**

A hashed page table is used in systems with **large virtual address spaces** to speed up address translation.

🔹 **Working:**

1. Virtual page number is passed to hash function
2. Hash function maps it to an index
3. Entry contains:
    - Page number
    - Frame number
4. If collision → use linked list

🔹 **Advantages:**

- Faster lookup
- Efficient for large memory systems

🔹 **Disadvantages:**

- Collision handling required

✅ **Conclusion:**

Hashed page tables improve performance in large systems.

🔁 **OR**

Dining Philosopher Problem & Solution

📌 **Answer:**

🔹 **Problem:**

Five philosophers share forks. Each needs two forks to eat.

👉 If all pick one fork → **deadlock occurs**

🔹 **Issues:**

- Deadlock
- Starvation

🔹 **Solutions:**

1. **Semaphore Solution**
   Control access to forks
2. **Limit Philosophers**
   Allow only 4 philosophers
3. **Message Passing**
   Request permission before eating

✅ **Conclusion:**

Proper synchronization prevents deadlock and starvation.

✅ **Q3(A). What is a Resource Allocation Graph? How does it help in deadlock detection?**

📌 **Answer:**

🔹 **Definition:**

A Resource Allocation Graph (RAG) is a graphical representation of processes and resources.

🔹 **Components:**

- **Processes** → circles
- **Resources** → squares
- **Edges:**
  - Request edge
  - Allocation edge

🔹 **Deadlock Detection:**

- If **cycle exists → deadlock possible**
- If no cycle → no deadlock

✅ **Conclusion:**

RAG helps visualize and detect deadlocks easily.

✅ **Q3(B). How can breaking the hold-and-wait condition prevent deadlocks?**

📌 **Answer:**

🔹 **Hold and Wait Condition:**

Process holds one resource and waits for another.

🔹 **Prevention Methods:**

1. **Request All Resources at Once**
   → No waiting while holding

2. **Release Before Requesting New**
   → Process must free resources

🔹 **Effect:**

- Eliminates circular waiting
- Prevents deadlock

❗ **Limitation:**

- May reduce resource utilization

✅ **Conclusion:**

Breaking hold-and-wait is an effective way to prevent deadlocks.

🔁 **OR**

Reader–Writer Problem using Semaphores

📌 **Answer:**

🔹 **Problem:**

- Multiple readers can read simultaneously
- Writers require exclusive access

🔹 **Solution using Semaphores:**

```
wait(mutex);
read_count++;
if(read_count == 1)
  wait(rw_mutex);
signal(mutex);

// Reading section
```

```
wait(mutex);
read_count--;
if(read_count == 0)
   signal(rw_mutex);
signal(mutex);
```

🔹 **Explanation:**

- First reader locks writer
- Last reader releases writer
- Writers get exclusive access

✅ **Conclusion:**

Ensures **safe concurrent reading and exclusive writing**.